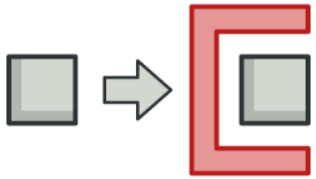
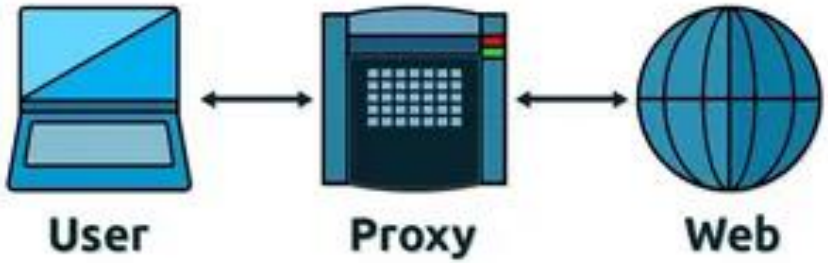
Структурные паттерны. Заместитель



Паттерн Заместитель
Proxy (Surrogate, Суррогат)



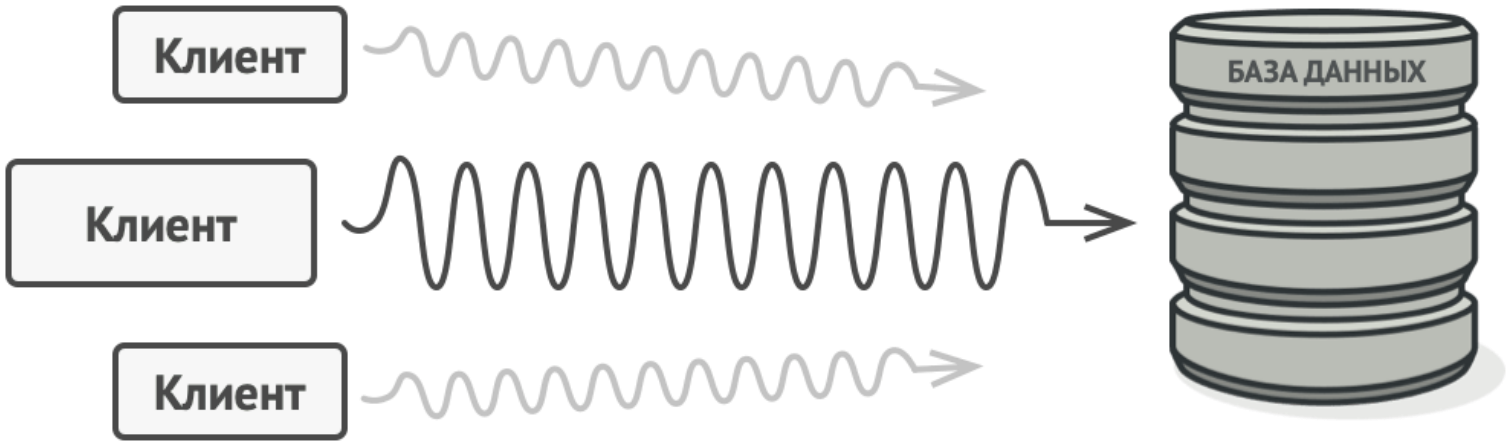
Заместитель — это структурный паттерн проектирования, который позволяет подставлять вместо реальных объектов специальные *объекты-заменители.*

Заместитель — это объект, который выступает посредником между *клиентом* и реальным сервисным объектом (*сервером*). Заместитель получает вызовы от клиента, выполняет свою функцию (контроль доступа, кеширование, логирование, изменение запроса и прочее), а затем передаёт вызов сервисному объекту.

Заместитель имеет тот же интерфейс, что и реальный объект, поэтому для клиента нет разницы — работать через заместителя или напрямую.

Проблема

Множество запросов клиентов к БД (или любому другому разделяемому ресурсу) нуждаются в упорядочивании, проверке прав доступа, регистрации, контроле продолжительности сеанса доступа и т.п.



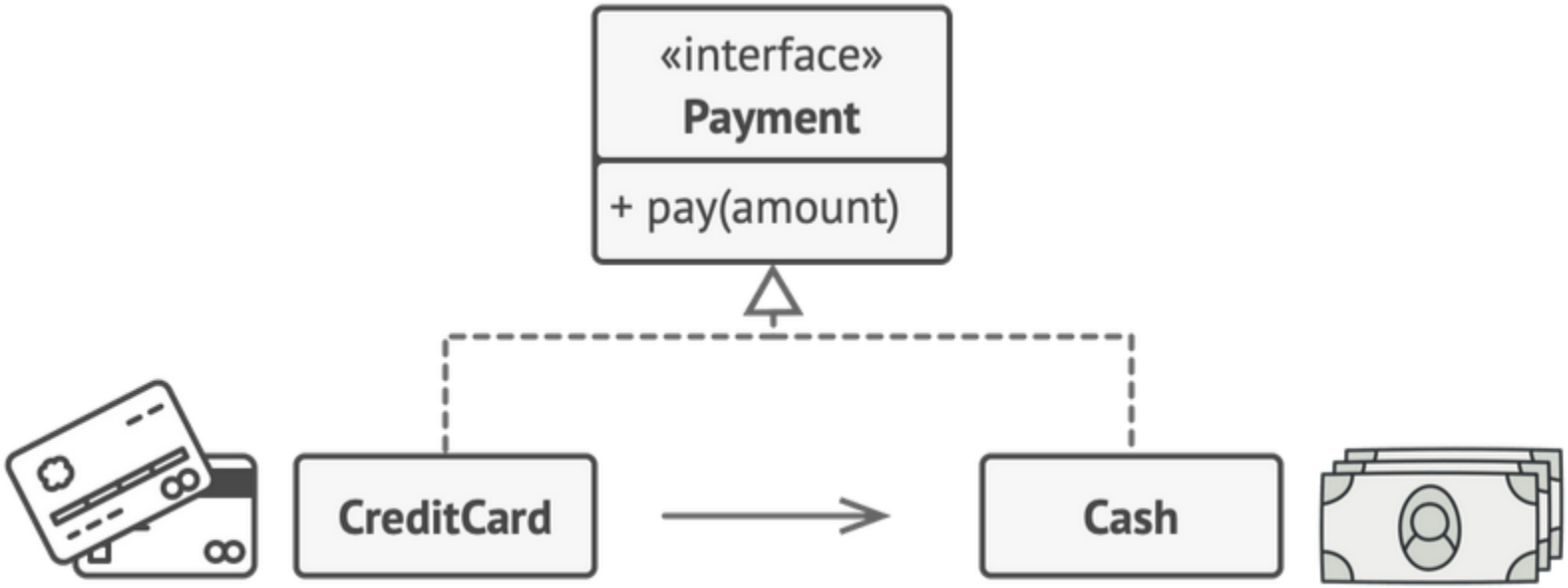
Решение

Паттерн **Заместитель** предлагает создать новый класс-дублёр, имеющий тот же *интерфейс*, что и оригинальный служебный объект.
При получении запроса от клиента объект-заместитель создает экземпляр служебного объекта и переадресовывает ему всю реальную работу.



В класс заместителя можно поместить какую-то промежуточную логику, которая выполнялась бы до (или после) вызовов этих же методов в настоящем объекте.
Благодаря одинаковому интерфейсу, объект-заместитель можно передать в любой код, ожидающий сервисный объект.

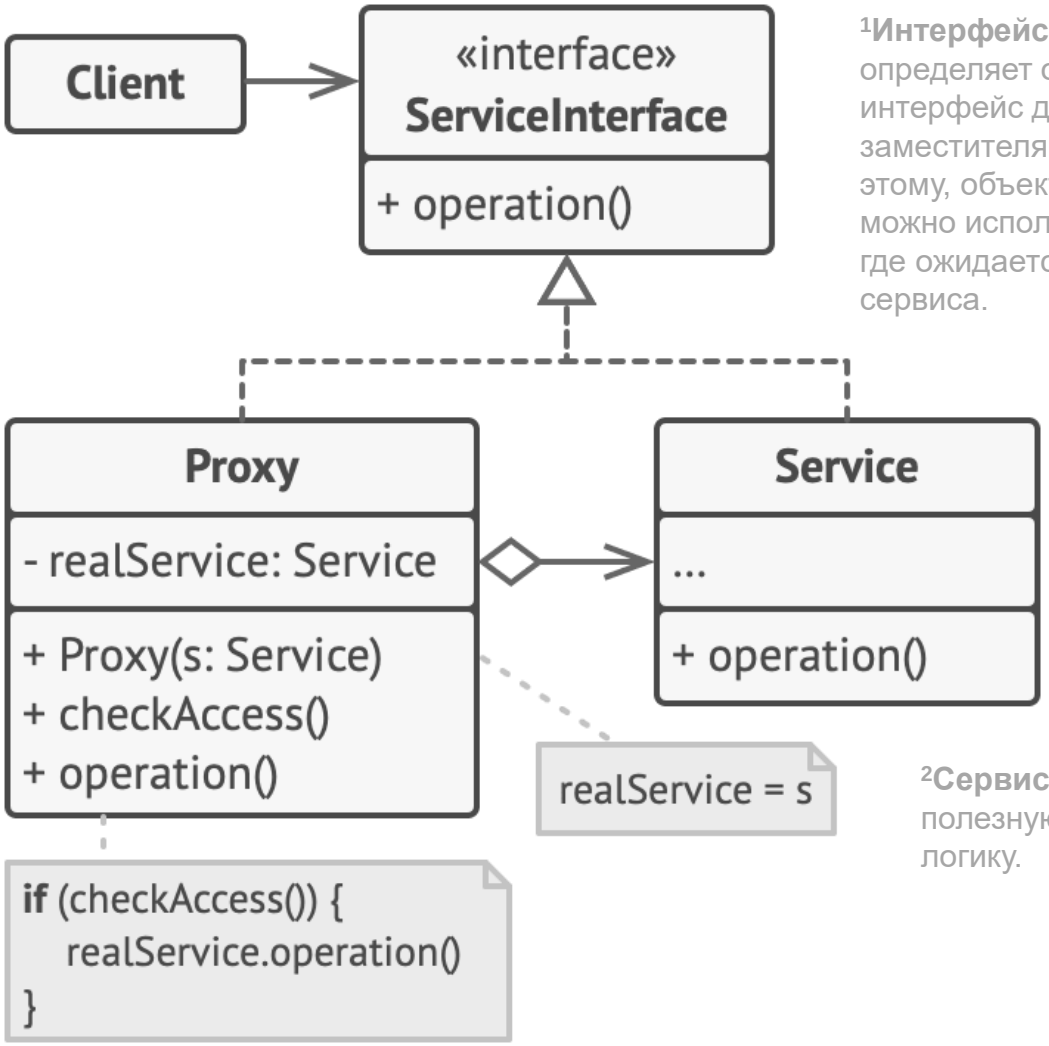
Решение



Платёжной картой можно расплачиваться, как и наличными.

Структура паттерна

[4]**Клиент** работает с объектами через интерфейс сервиса. Благодаря этому он не замечает подмены сервиса заместителем



[1]**Интерфейс сервиса** определяет общий интерфейс для сервиса и заместителя. Благодаря этому, объект заместителя можно использовать там, где ожидается объект сервиса.

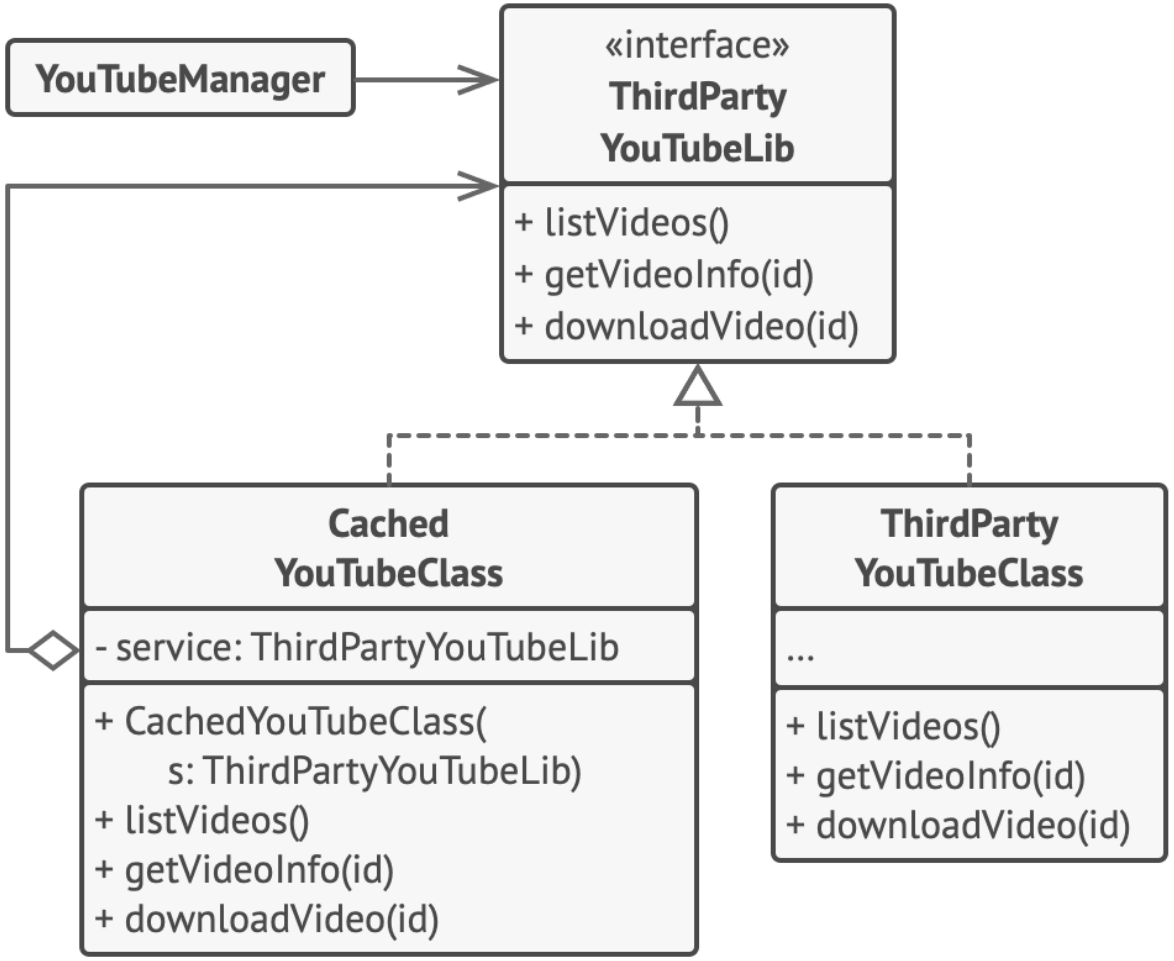
[3]**Заместитель** хранит ссылку на объект сервиса. После того как заместитель заканчивает свою работу (например, инициализацию, логирование, защиту или другое), он передаёт вызовы вложенному сервису.

Заместитель может сам отвечать за создание и удаление объекта сервиса.

[2]**Сервис** содержит полезную бизнес-логику.

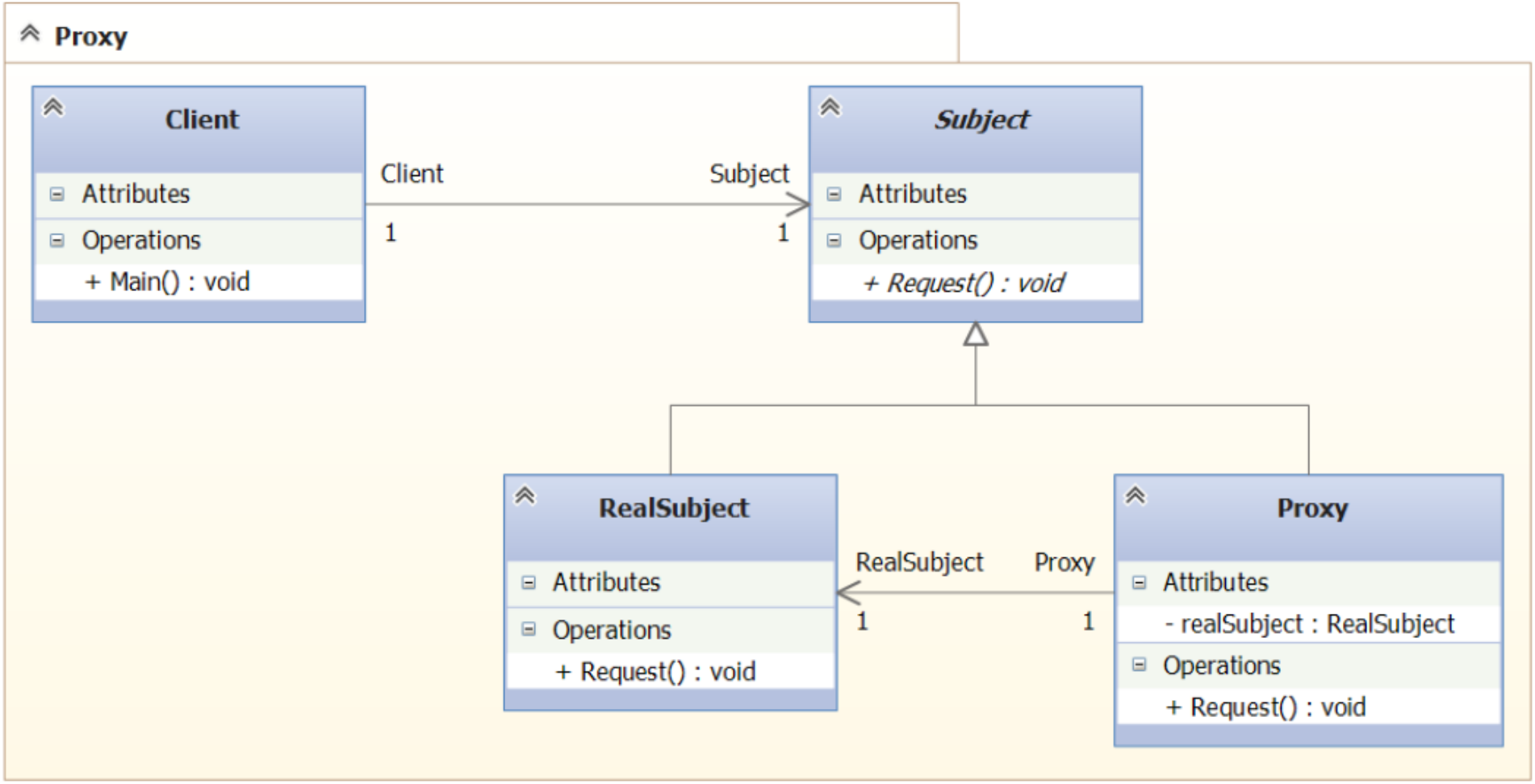
Пример

Кеширование работы с видеохостингом



- *Оригинальный* объект выполняет загрузку по сети, даже если пользователь запрашивал одно и то же видео.
- **Заместитель** загружает видео только один раз, используя для этого служебный объект; в остальных случаях возвращает кешированный файл.

Структура паттерна UML



Структура паттерна на языке C#

Subject - **Субъект**:
Предоставляет общий интерфейс для Proxy и RealSubject. Proxy возможно использовать везде где ожидается использование RealSubject.

```
class Program
{
    static void Main()
    {
        Subject subject = new Proxy();
        subject.Request();
    }
}
```

```
abstract class Subject
{
    public abstract void Request();
}
```

```
class RealSubject : Subject
{
    public override void Request()
    {
        Console.WriteLine("RealSubject");
    }
}
```

```
class Proxy : Subject
{
    RealSubject realSubject;

    public override void Request()
    {
        if (realSubject == null)
            realSubject = new RealSubject();

        realSubject.Request();
    }
}
```

RealSubject - **Реальный субъект**:
Представляет собой класс объекта, для которого требуется создание заместителя.

Proxy - **Заместитель**:
Представляет собой класс объекта-заместителя. Объект заместитель хранит в себе ссылку на реальный субъект, что позволяет заместителю обращаться к реальному субъекту напрямую. Заместитель имеет такой же интерфейс, как и реальный субъект, что позволяет в нужный момент подставлять заместителя вместо реального субъекта и наоборот. Заместитель контролирует доступ к реальному субъекту и может отвечать за создание экземпляра реального субъекта если это может оказаться необходимым.

Отношения между участниками

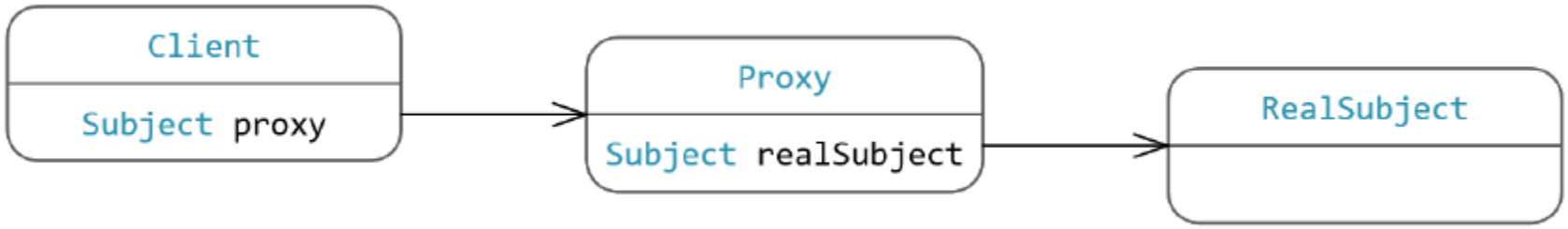
Отношения между классами

- Класс RealSubject связан связью отношения наследования с абстрактным классом Subject.
- Класс Proxy связан связью отношения наследования с абстрактным классом Subject и связью отношения ассоциации с классом RealSubject.

Отношения между объектами

- Объект-заместитель класса Proxy, переадресует клиентские запросы целевому объекту класса RealSubject.

Диаграмма объектов



Применение

Ленивая инициализация (*виртуальный прокси*). Когда есть тяжёлый объект, грузящий данные из файловой системы или базы данных.

Вместо того, чтобы грузить данные сразу после старта программы, можно сэкономить ресурсы и создать объект тогда, когда он действительно понадобится.

Защита доступа (*защищающий прокси*). Когда в программе есть разные типы пользователей, и необходимо защищать объект от неавторизованного доступа.

Прокси может проверять доступ при каждом вызове и передавать выполнение служебному объекту, если доступ разрешён.

Локальный запуск сервиса (*удалённый прокси*). Когда настоящий сервисный объект находится на удалённом сервере.

В этом случае заместитель транслирует запросы клиента в вызовы по сети в протоколе, понятном удалённому сервису.

Логирование запросов (*логирующий прокси*). Когда требуется хранить историю обращений к сервисному объекту.

Заместитель может сохранять историю обращения клиента к сервисному объекту.

Кеширование объектов *(«умная» ссылка*). Когда нужно кешировать результаты запросов клиентов и управлять их жизненным циклом.

Заместитель может подсчитывать количество ссылок на сервисный объект, которые были отданы клиенту и остаются активными. Когда все ссылки освобождаются, можно будет освободить и сам сервисный объект (например, закрыть подключение к базе данных).

Шаги реализации

1. **Определить интерфейс,** который бы сделал заместитель и оригинальный объект взаимозаменяемыми.
2. **Создать класс заместителя.** Он должен содержать ссылку на сервисный объект. Чаще всего, сервисный объект создаётся самим заместителем. В редких случаях заместитель получает готовый сервисный объект от клиента через конструктор.
3. **Реализовать методы заместителя** в зависимости от его предназначения. В большинстве случаев, проделав какую-то полезную работу, методы заместителя должны передать запрос сервисному объекту.
4. **Проанализировать необходимость введения фабрики**, которая решала бы, какой из объектов создавать — *заместитель* или *реальный сервисный объект*. Эта логика, также, может быть помещена в создающий метод самого заместителя.
5. **Продумать необходимость реализации «ленивой» инициализации** сервисного объекта при первом обращении клиента к методам заместителя.

Преимущества и недостатки

+ Позволяет контролировать сервисный объект незаметно для клиента.
+ Может работать, даже если сервисный объект ещё не создан.
+ Может контролировать жизненный цикл служебного объекта.
- Усложняет код программы из-за введения дополнительных классов.
- Увеличивает время отклика от сервиса.